ChatGPT
New chat
Images
Library
Scheduled
Plugins
Projects
Codex
More
Recents
Generate Interview Questions
Explain Web APIs
Redux Toolkit Path
Clean Up Jinja Template
Kalpesh Vasan
Free
Upgrade
Upgrade
Share
Ask anything
Think

i want 50 machine coding questions for frontend interview for 8 yrs of experirnce

Absolutely. For **8 years of frontend experience**, machine-coding interviews should go beyond simple "build a Todo app." You should practice **component architecture, state management, async behavior, performance, accessibility, reusable components, edge cases, and scalable UI design.**

Here are **50 machine-coding questions**, arranged from intermediate → senior → architect-level.

### 1. JavaScript + DOM Fundamentals

1. **Autocomplete / Search Suggestions**
   - Debouncing
   - API integration
   - Loading/error/empty states
   - Keyboard navigation
   - Request cancellation
2. **Infinite Scroll**
   - `IntersectionObserver`
   - Pagination
   - Loading states
   - Duplicate prevention
   - Error retry
3. **Custom Debounce & Throttle Utility**
   - Implement from scratch
   - `leading` / `trailing`
   - `cancel()`
   - `flush()`
4. **Countdown Timer**
   - Start/pause/resume/reset
   - Accurate timing
5. **Event Emitter**
   - `on`
   - `off`
   - `once`
   - `emit`
   - Multiple listeners
6. **Custom Promise Implementation**
   - `then`
   - `catch`
   - `finally`
   - Chaining
   - Error propagation
7. **Retry API Utility**
   - Retry failed requests
   - Exponential backoff
   - Maximum retries
   - Cancellation
8. **Parallel API Request Manager**
   - Concurrency limit
   - Queue management
   - Success/failure handling
9. **File Upload Component**
   - Progress
   - Multiple files
   - Cancel upload
   - Retry
   - Validation
10. **Drag-and-Drop File Upload**
- Drag events
- File validation
- Preview
- Upload progress

---

### 2. React Machine Coding

11. **Todo Application**
- Add/edit/delete
- Complete/uncomplete
- Filtering
- Search
- LocalStorage
12. **Advanced Todo Application**
- Categories
- Priority
- Due dates
- Sorting
- Pagination
- Persistent state
13. **Accordion Component**
- Single/multiple open
- Controlled/uncontrolled
- Keyboard accessibility
- Animation
14. **Tabs Component**
- Dynamic tabs
- Active tab
- Keyboard navigation
- Lazy-loaded content
15. **Modal Component**
- Portal
- ESC close
- Outside click
- Focus management
- Nested modal handling
16. **Tooltip Component**
- Hover
- Focus
- Positioning
- Viewport boundary detection
17. **Dropdown / Select**
- Search
- Multi-select
- Keyboard navigation
- Select all
- Clear selection
18. **Multi-Select Component**
- Chips
- Search
- Keyboard navigation
- Async options
- Maximum selection
19. **Date Picker**
- Calendar
- Month/year navigation
- Date range
- Disabled dates
- Keyboard support
20. **Toast Notification System**
- Success/error/info/warning
- Auto-dismiss
- Queue
- Pause on hover
- Global API

---

### 3. Advanced UI Components

21. **Data Table**
- Sorting
- Filtering
- Pagination
- Column selection
- Row selection
22. **Advanced Data Grid**
- Virtual scrolling
- Sorting
- Filtering
- Column resizing
- Sticky columns
- Row selection
23. **Virtualized List**
- Render thousands of records
- Windowing
- Dynamic item height
- Scroll position management
24. **Tree View**
- Expand/collapse
- Nested nodes
- Checkbox selection
- Select parent/children
- Lazy loading
25. **File Explorer**
- Folder navigation
- Create/delete/rename
- Tree structure
- Context menu
- Drag and drop
26. **Kanban Board**
- Columns
- Cards
- Drag and drop
- Add/edit/delete
- Persistent state
27. **Nested Comment System**
- Reddit-style comments
- Reply
- Edit/delete
- Collapse threads
- Pagination
28. **Pagination Component**
- Page navigation
- Page size
- First/last
- Ellipsis
- URL synchronization
29. **Carousel / Image Slider**
- Next/previous
- Auto-play
- Pause on hover
- Touch/swipe
- Lazy loading
30. **Image Gallery**
- Grid
- Lightbox
- Zoom
- Keyboard navigation
- Lazy loading

---

### 4. State Management / Redux

31. **Shopping Cart**
- Add/remove products
- Quantity
- Price calculation
- Coupons
- Persistence
32. **E-commerce Product Listing**
- Search
- Filters
- Sorting
- Pagination
- Cart
- Wishlist
33. **Redux-Based Authentication**
- Login/logout
- Protected routes
- Token handling
- Refresh token flow
- User state
34. **Global Notification System**
- Redux store
- Queue
- Priority
- Auto-dismiss
- Duplicate prevention
35. **Multi-Step Form**
- Redux state
- Validation
- Previous/next
- Save draft
- Resume later
36. **Shopping Cart with Optimistic Updates**
- Optimistic UI
- API failure rollback
- Loading states
- Error handling

---

### 5. Real-World Product Features

37. **Chat Application**
- Conversations
- Send message
- Typing indicator
- Read/unread
- Infinite history
- Optimistic messages
38. **Notification Center**
- Read/unread
- Mark all read
- Infinite scroll
- Group notifications
- Real-time updates
39. **Dashboard**
- Multiple widgets
- Drag/drop widgets
- Resize widgets
- API data
- Loading/error states
- Persist layout
40. **Admin Dashboard**
- Sidebar
- Routing
- Tables
- Filters
- Charts
- Role-based access
41. **Search Engine UI**
- Search
- Suggestions
- Recent searches
- Filters
- Pagination
- URL synchronization
42. **Product Comparison**
- Select products
- Comparison table
- Dynamic attributes
- Remove products
- Responsive design
43. **Booking Application**
- Calendar
- Available slots
- Select time
- Booking confirmation
- API integration
44. **Form Builder**
- Add fields
- Remove fields
- Reorder fields
- Field configuration
- Preview form
- Validation
45. **Permission Management UI**
- Users
- Roles
- Permissions
- Nested permissions
- Select all/indeterminate state
- Save changes

---

### 6. Senior / 8-Year Experience Challenges

These are the ones I'd give **extra priority** for an 8-year frontend candidate.

46. **Build a Mini React Query**

Implement a simplified data-fetching library:

```
useQuery()
useMutation()
cache
staleTime
refetch
retry
loading/error/success
request deduplication
```

This tests whether you understand **async state architecture**, not just React syntax.

47. **Build a Mini Redux**

Implement:

```
createStore()
dispatch()
subscribe()
getState()
combineReducers()
middleware()
```

Then build an application using it.

48. **Build a Design System**

Create reusable:

```
Button
Input
Select
Modal
Tooltip
Tabs
Accordion
Toast
Table
Pagination
```

Requirements:

- Consistent API
- Accessibility
- Theme support
- Controlled/uncontrolled patterns
- Reusable hooks
- Composition

49. **Build a High-Performance Data Dashboard**

Imagine:

```
100,000+ records
```

Requirements:

- Virtualization
- Server-side filtering
- Server-side sorting
- Pagination
- Debounced search
- Caching
- Web Workers where appropriate
- Error handling
- Loading states

The interviewer should be able to ask:

> "Why is your implementation performant?"

50. **Build a Mini E-commerce Application**

This is a great **final senior-level machine-coding project**.

Requirements:

```
Product Listing
     ↓
Search
     ↓
Filters
     ↓
Sorting
     ↓
Product Details
     ↓
Cart
     ↓
Checkout
```

Add:

- Redux Toolkit
- API integration
- Authentication
- Protected routes
- Optimistic updates
- Pagination
- Caching
- Error boundaries
- Lazy loading
- Code splitting
- Accessibility
- Responsive UI

---

### ⭐ What I would prioritize for an 8-year candidate

Don't spend equal time on all 50.

| Priority | Questions |
|---|---|
| 🔥 Must Do | 1, 2, 15, 17, 21, 23, 24 |
| 🔥 Senior | 26, 27, 31, 35, 36, 37 |
| 🔥 Very Senior | 39, 40, 44, 45, 46 |
| 🚀 Architect Level | 47, 48, 49, 50 |

#### The most important 15

If your interview is coming soon, I'd practice these first:

1. Autocomplete
2. Infinite Scroll
3. Debounce/Throttle
4. Modal
5. Dropdown
6. Multi-select
7. Data Table
8. Virtualized List
9. Tree View
10. Kanban Board
11. Shopping Cart
12. Multi-step Form
13. Chat Application
14. Form Builder
15. Mini E-commerce

#### What interviewers will evaluate at 8 years

The important difference is that they usually aren't evaluating only **"Can you make it work?"**

They'll look for:

```
Correctness
   ↓
Component Design
   ↓
State Management
   ↓
Reusability
   ↓
Accessibility
   ↓
Performance
   ↓
Error Handling
   ↓
Testing
   ↓
Scalability
   ↓
Trade-offs / Architecture
```

So for every machine-coding problem, practice explaining **why you designed it that way**, not just writing the code.

If you're preparing specifically for a **Senior Frontend / Lead Frontend role**, I can also ↪ turn these 50 problems into a **30-day machine-coding preparation plan**, with **Day → Question → requirements → expected folder structure → hints → follow-up interview questions → solution**, which would be much more useful than simply solving 50 unrelated problems.

ChatGPT can make mistakes. Check important info.